

实验 - 使用 API 模拟器和 Postman 探索 Rest API

目标

- 第 1 部分：启动 DEVASC 虚拟机
- 第 2 部分：使用 API 模拟器探索 API 文档
- 第 3 部分：使用 Postman 对 API 模拟器进行 API 调用
- 第 4 部分：使用 Python 将 100 本书添加到 API 模拟器

背景/场景

DEVASC 虚拟机包括一个带 API 文档和一个相关数据库的 School Library API 模拟器。您可以离线使用模拟器来浏览 API 并测试它们的功能。

在本实验中，您将学习如何使用 School Library API 模拟器进行 API 调用以列出、添加和删除图书。稍后，您将使用 Postman 来进行这些相同的 API 调用。

所需资源

- 1 台带有您选择的操作系统的 PC
- Virtual Box 或 VMWare
- DEVASC 虚拟机

说明

第 1 部分： 启动 DEVASC 虚拟机

如果您尚未完成“实验 - 安装虚拟机实验室环境”，请立即完成。如果您已完成该实验，请立即启动 DEVASC 虚拟机。

第 2 部分： 使用 API 模拟器浏览 API 文档

为了了解如何调用 REST API，开发人员通常首先研究 API 文档。REST API 的请求、响应、头部和参数的格式通常使用 OpenAPI 规范（以前的 Swagger 规范）进行记录。

步骤 1： 打开 Chromium Web 浏览器。

双击桌面上的 Chromium Web 浏览器图标。

步骤 2： 连接到学校图书馆网站。

如果浏览器没有自动打开学校图书馆网站，请在地址栏键入：**libraryary.demo.local**，然后按回车键。

步骤 3： 转到 API 文档页面。

- 网站默认为“**Our Books**”选项卡，并显示图书列表。在右上角，其中显示，**Click here for API docs**，单击 **here** 转到 API 文档网页。

现在，您将看到 **/api/v1 Default namespace** 中的 API 列表。

- b. 请注意最右侧的向下箭头。单击 **/api/v1** 栏上的任意位置将最小化 API 列表并将箭头向右转。再次单击同一栏以重新显示 API 列表。

请注意几个 API 最右侧的锁。锁表示这些 API 需要使用令牌。

步骤 4: 使用 GET /books API 列出图书。

单击 **GET /books** API 栏上的任意位置。此 API 返回学校图书馆中的图书列表。

- **参数** - 有几个可选的 API 参数。这些选项可用于对输出进行过滤、排序或分页。本实验稍后将参考这些内容。
- **响应内容类型** - 单击 **application/json** 可查看可查看信息的不同类型数据格式的列表。将选择保留为 **application/json**。
- **代码** - 默认情况下，代码显示 200，表示来自服务器的 API 请求成功，如说明中所示。（您尚未发送 API 请求。）

步骤 5: 使用 API 文档中的试用功能。

OpenAPI 规范更强大的功能之一是能够测试 API 调用以查看是否正确构建了它。您还可以查看响应以查看响应是否符合您的期望。您将在适用于思科、MapQuest 和其他使用此 OpenAPI 规范功能的组织的 API 文档中看到此相同的测试功能。

- a. 在 **GET /books** API 文档中，单击 **Try it out** 按钮。
- b. 请注意，您现在可以选择输入可选参数的信息。将参数留空，然后单击“**Execute**”按钮。

在“**Responses**”部分，您将看到：

- **Curl:** 可用于访问 /books API 相同信息的 curl 命令。
- **请求 URL:** 此 URL 用于 API 请求，可用于使用 curl、Postman 和 Python 请求相同的信息。
- **代码:** 这是 HTTP 响应代码。200 表示调用成功。
- **响应正文:** JSON 格式的书籍列表。
- **响应头部:** 有关从服务器返回的 API 的信息。

在 **响应正文** 中，您将看到 JSON 格式的书籍列表：

```
[
{
  "id": 0,
  "title": "IP Routing Fundamentals",
  "author": "Mark A. Sportack"
},
{
  "id": 1,
  "title": "Python for Dummies",
  "author": "Stef Maruch Aahz Maruch"
},
{
  "id": 2,
  "title": "Linux for Networkers",
  "author": "Cisco Systems Inc."
}
```

```
},  
{  
  "id": 3,  
  "title": "NetAcad: 20 Years Of Online-Learning",  
  "author": "Cisco Systems Inc."  
}  
]
```

步骤 6: 在终端窗口中使用 curl 命令。

GET /books API 提供了使用 curl 访问响应正文中显示的内容的信息。curl 是一种命令行工具，用于使用任何受支持的协议（包括 HTTP 和 HTTPS）向服务器传输数据或从服务器传输数据。

- a. 选择 curl 命令，右键单击并将其复制到剪贴板：

```
curl -X GET "http://library.demo.local/api/v1/books" -H "accept:  
application/json"
```

打开终端窗口。右键单击并将剪贴板中的内容 **粘贴** 到终端，然后按回车键。请注意，这里提供的信息与库的 OpenAPI 接口相同。

```
devasc@labvm:~$ curl -X GET "http://library.demo.local/api/v1/books" -H  
"accept: application/json"  
[  
  {  
    "id": 0,  
    "title": "IP Routing Fundamentals",  
    "author": "Mark A. Sportack"  
  },  
  {  
    "id": 1,  
    "title": "Python for Dummies",  
    "author": "Stef Maruch Aahz Maruch"  
  },  
  {  
    "id": 2,  
    "title": "Linux for Networkers",  
    "author": "Cisco Systems Inc."  
  },  
  {  
    "id": 3,  
    "title": "NetAcad: 20 Years Of Online-Learning",  
    "author": "Cisco Systems Inc."  
  }  
]  
devasc@labvm:~$
```

步骤 7: 使用 GET /books API 列出具有 ISBN 的图书。

- a. 返回学校图书馆 API 网站的 GET /books API。
- b. 在“**Parameters**”部分中，选择 **includeISBN** 参数旁边的向下箭头，然后选择 **true**。
- c. 点击 **Execute**

请注意 **Responses** 中的以下变化：

- **Curl** 现在包括 ISBN 的参数。

```
curl -X GET "http://library.demo.local/api/v1/books?includeISBN=true" -H
"accept: application/json"
```

- **请求 URL** 现在包含 ISBN 的参数。

```
http://library.demo.local/api/v1/books?includeISBN=true
```

- **响应正文** 具有与之前所示相同的图书列表，但现在包括该图书的 ISBN。

为了最大限度地减少滚动，当您完成 API 时，您可以通过单击标题栏上的任意位置关闭该特定的 API 窗口。现在，您可以更轻松地查看所有 API。

步骤 8： 使用 POST/loginViaBasic API 获取令牌。

- 单击 API **POST /loginViaBasic**。
- 请注意，没有参数。单击 **Try it out**，然后单击 **Execute**。
- 会有一个 **Sign in** 框提示您输入 **用户名** 和 **密码**。输入以下信息并单击 **Sign in**：
 - **用户名**：cisco
 - **密码**：Cisco123!
- 令牌将显示在 **响应正文** 中。选择引号之间的信息，右键单击并将信息 **复制** 到剪贴板。您的令牌将与下面显示的令牌不同。

```
{
  "token": "cisco|KZzZtteQbC5iV3HKEzB7hCJ6qHQXen4rLGh72YJKeVfs"
}
```

- 向上滚动到 School Library API 页面的顶部，然后单击绿色的 **Authorize** 按钮。将出现 “**Available authorizations**” 对话框。
- 右键单击并在 **Value** 后 **粘贴** 令牌，然后单击 **Authorize**。请注意，**名称** 是 X-API-KEY。此信息以及令牌将在稍后的 Postman 中使用。
- 关闭 “**Available authorizations**” 对话框并返回到 API 列表。请注意，几个 API 的锁定现在已经更改。这些 API 现在可供您使用。
- 单击 API **POST /loginViaBasic** 以关闭窗口。

步骤 9： 使用 POST /books API 添加图书。

- 单击 API **POST /books**。
- 在 “**Parameters**” 下注意 **有效载荷** 是必需的。这意味着此 API 需要 **Parameter content type**（即 JSON）指定的格式提供此参数的信息。
- 单击 **Try it out**。
- 使用下面显示的信息修改 **id**、**title** 和 **author**。

```
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani"
}
```

- e. 点击**执行 (Execute)**
- f. 验证该推送在服务器响应中是否成功。**代码** 为 200 表示推送是成功的。您应该看到您在 **响应正** 文中添加的电子书以及一个新的 **id**。您还将看到 **curl** 和 **请求 URL** 的更新信息。
- g. 要添加另一本书，请使用以下信息修改 **id**、**title** 和 **author** 。

```
{
  "id": 5,
  "title": "31 Days Before Your CCNA Exam",
  "author": "Allan Johnson"
}
```
- h. 点击**执行 (Execute)**
- i. 验证该推送在 **服务器响应** 中是否成功。**代码** 为 200 表示推送是成功的。您应该看到您在 **响应正** 文中添加的电子书以及一个新的 **id**。您还将看到 **curl** 和 **请求 URL** 的更新信息。

注意：如果您看到代码为 **401**，请检查 **响应正** 文文本。您很可能收到“**错误**”：“无效的 API 密钥”响应。这是因为您没有为 API 密钥输入所有字符。或者，你可能添加了一个不必要的空间。返回上一步并重复授权过程。
- j. 单击 **API POST/书籍** 栏以关闭窗口。
- k. 您可以验证图书是否已添加到“**Our Books**”页面。返回浏览器中的“**School Library**”选项卡 (<http://library.demo.local>) 并刷新页面。注意不要关闭 School Library API 选项卡。如果你这样做，那么你将需要重新验证。

步骤 10: 使用 GET /books API 列出图书。

- a. 返回浏览器中的 **School Library API** 选项卡。单击 **GET /books API**。
- b. 单击 **Try it out**。如果您看到红色的“**Cancel**”按钮，则您已处于“**Try it out**”模式。
- c. 点击 **Execute**。
- d. 在 **响应正文中的服务器响应**下，您现在将看到您添加的两本书。请注意，每本书都有一个唯一的 **id**。

```
[
  {
    "id": 0,
    "title": "IP Routing Fundamentals",
    "author": "Mark A. Sportack"
  },
  {
    "id": 1,
    "title": "Python for Dummies",
    "author": "Stef Maruch Aahz Maruch"
  },
  {
    "id": 2,
    "title": "Linux for Networkers",
    "author": "Cisco Systems Inc."
  },
  {
    "id": 3,
```

```
"title": "NetAcad: 20 Years Of Online-Learning",
"author": "Cisco Systems Inc."
},
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani"
},
{
  "id": 5,
  "title": "31 Days Before Your CCNA Exam",
  "author": "Allan Johnson"
}
]
```

- e. 单击 **GET /books** API 以关闭窗口。

步骤 11: 使用 **GET /books{id}** API 列出特定图书。

- 单击 **GET /books{id}** API。请注意，此 API 需要 **id** 作为参数。
- 在“Parameters”右侧，单击“Try it out”按钮。
- 在“Parameters”下，输入 4 作为所需的 id。
- 单击 **Execute**。请注意 **curl** 和 **请求 URL** 提供的信息。
 - Curl** - 这是使用 curl 执行相同功能的 curl 命令。
 - 请求 URL** - 这是可用于使用 Postman 和 Python 获取相同信息的 URL。

验证获取在 **服务器响应**中是否成功。**代码** 为 200 表示推送是成功的。在 **响应正文** 中，您将看到您请求的 **id** 为 4 的电子书。

```
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani"
}
```

- e. 单击 **GET /books{id}** API 以关闭窗口。

步骤 12: 使用 **DELETE /books{id}** API 删除特定图书。

- 单击 **DELETE /books{id}** API。请注意，此 API 需要 **id** 作为参数。
- 单击 **Try it out**。
- 在“Parameters”下，输入 4。
- 单击 **Execute**。
- 在 **服务器响应**中验证删除是否成功。**代码** 为 200 表示推送是成功的。在 **响应正文** 中，您将看到您删除的 **id** 为 4 的电子书。

```
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani"
}
```

```
}
```

- f. 单击 **DELETE /books{id}** API 以关闭窗口。

步骤 13: 使用 GET /books API 列出图书。

- 单击 **GET /books** API。
- 单击 **Try it out**。如果您看到红色的“**Cancel**”按钮，则您已处于“**Try it out**”模式。
- 单击**执行 (Execute)**
- 在 **响应正文**中的**服务器响应**下，您将不再看到 **id** 为 4 的电子书。

```
[
  {
    "id": 0,
    "title": "IP Routing Fundamentals",
    "author": "Mark A. Sportack"
  },
  {
    "id": 1,
    "title": "Python for Dummies",
    "author": "Stef Maruch Aahz Maruch"
  },
  {
    "id": 2,
    "title": "Linux for Networkers",
    "author": "Cisco Systems Inc."
  },
  {
    "id": 3,
    "title": "NetAcad: 20 Years Of Online-Learning",
    "author": "Cisco Systems Inc."
  },
  {
    "id": 5,
    "title": "31 Days Before Your CCNA Exam",
    "author": "Allan Johnson"
  }
]
```

注意： 不要关闭 **Chromium** 浏览器中的 **School Library API** 选项卡。您将在下一部分中使用 API 文档。

第 3 部分: 使用 Postman 对 API 模拟器进行 API 调用

在本部分中，您将使用 **Postman** 来执行您在学生库 API 文档中进行的相同 API 调用。当 API 开发人员网站不可用时，**Postman** 是一个有用的工具，同时提供了轻松保存、组织和重复使用 API 的功能。

步骤 1: 打开 Postman。

双击桌面上的 **Postman** 图标。通常情况下，您会登录 **Postman**。但是，没有必要在本课程中获取帐户并登录到 **Postman** 实验室。

步骤 2: 使用 GET /books API 列出图书。

- a. 在 **Launchpad** 选项卡旁边的主窗口中，单击加号图标 “+” 以创建 **无标题请求**。默认情况下，这将是 **GET** 请求。
- b. 单击 **GET** 旁边的向下箭头可查看不同的 API 操作，包括 GET、POST 和 DELETE。在 **GET** 上保留选择。单击 **GET** 旁边的向上箭头关闭列表。
- c. 输入请求 URL。
 - 1) 返回到 Chromium 中的 **School Library API** 选项卡，如有必要，展开 **GET /books API**。
 - 2) 在 “**请求 URL**” 下，选择，右键单击并将 **URL 复制** 到剪贴板：
`http://library.demo.local/api/v1/books`
 - 3) 返回 **Postman** 并在 GET 旁边粘贴 URL，其中显示 “Enter request URL”。

注意：如果粘贴时在 URL 下方添加了一行，请删除额外的行。
- d. 单击**发送**。要验证 API 请求是否成功，您现在将看到一个响应，其中包含绿色 **Status** 代码 200 OK。向下滚动到 “**正文**” 部分以查看响应。请注意，默认值是 **Pretty** 和 **json**。

```
[
{
  "id": 0,
  "title": "IP Routing Fundamentals",
  "author": "Mark A. Sportack"
},
{
  "id": 1,
  "title": "Python for Dummies",
  "author": "Stef Maruch Aahz Maruch"
},
{
  "id": 2,
  "title": "Linux for Networkers",
  "author": "Cisco Systems Inc."
},
{
  "id": 3,
  "title": "NetAcad: 20 Years Of Online-Learning",
  "author": "Cisco Systems Inc."
},
{
  "id": 5,
  "title": "31 Days Before Your CCNA Exam",
  "author": "Allan Johnson"
}
]
```

注意：您可以使用输出上方的 “Save Response” 按钮将 JSON 输出保存到文件 中。本实验不需要这样做。

步骤 3: 使用 POST/loginViaBasic API 获取令牌。

- a. 在主窗口中，单击加号图标 “+” 以创建新的 **无标题请求**。

- b. 单击 GET 旁边的向下箭头，然后选择 **POST**。
- c. 输入请求 URL。
 - 1) 返回到 Chromium 中的 **School Library API** 选项卡，如有必要，展开 **POST /loginViaBasic** API。
 - 2) 在“请求 URL”下，选择，右键单击并将 URL 复制到剪贴板：

`http://library.demo.local/api/v1/loginViaBasic`

注意：如果不再显示“请求 URL”，则您可能要关闭并重新打开“**School Library API** 文档”页面，并且不再进行身份验证。单击 **Try it out**，然后执行，然后使用用户名 `cisco` 和密码 `Cisco123!` 重新进行身份验证。

- 3) 返回 **Postman** 并在 POST 旁边粘贴 URL，其中显示“输入请求 URL”。

注意：如果粘贴时在 URL 下方添加了一行，请删除额外的行。

- d. 点击 **Authorization**。在此区域内，完成以下操作：
 - 1) 在“**Type**”下拉列表中，选择“**Basic Auth**”。
 - 2) 对于“用户名”和“密码”字段，填写以下内容：
 - 用户名：cisco
 - 密码：Cisco123!
- e. 单击**发送**。

- f. 如有必要，请向下滚动到“正文”部分以查看您的新令牌。您的令牌将与此处显示的令牌不同。

```
{
  "token": "cisco|5xSUHYFDvIAoCRv0LqWVSDcjJAwWjg18vMml6u2lm1I"
}
```

步骤 4：使用 POST /books API 添加图书。

现在，在使用 *School Library API* 文档中的 **Try it out** 功能时，您将添加在第 2 部分中删除的 **IPv6 Fundamentals** 一书。

- a. 在主窗口中，单击加号图标“+”以创建 **无标题请求**。
- b. 单击 GET 旁边的向下箭头，然后选择 **POST**。
- c. 输入请求 URL。
 - 1) 返回到 Chromium 中的 **School Library API** 选项卡，并展开 **POST /books** API。
 - 2) 在“请求 URL”下，选择，右键单击并将 URL 复制到剪贴板：

`http://library.demo.local/api/v1/books`

注意：如果请求 URL 不再显示，则您可能已取消 **Try it out**。单击 **试用**，然后单击**执行**以显示请求 URL。

- 3) 返回 **Postman** 并在 POST 旁边粘贴 URL，其中显示“输入请求 URL”。

注意：如果粘贴时在 URL 下方添加了一行，请删除额外的行。

- d. 点击 **Authorization**。在此区域内，完成以下操作：
 - 1) 在 **Type** 的下拉列表中，选择 **API Key**。
 - 2) 在 **Key** 字段中，输入 **X-API-KEY**。

注意：回想一下，当您有一个令牌选择绿色的授权按钮时，您在 School Library API 网页中看到了 X-API-KEY。

- 3) 返回 Postman 中的 “**Post**” 选项卡并复制您在步骤 3 中收到的令牌。请务必在引号内包含所有内容。您的令牌将与此处显示的令牌不同。

示例: `cisco|5xSUHYFDvIAoCRv0LqWVSDcjJAwWjg18vMm16u2lm1I`

- 4) 返回 Postman 中的第二个 **Post** 选项卡。将标记粘贴到 “**Value**” 字段中
- e. 在 “**Authorization**” 选项卡的同一行中，单击 “**正文**”。此部分将允许您选择输入的格式。
- 单击 **raw** 单选按钮。
 - 单击 **Text** 并将此选项更改为 **JSON**。
- f. 在输入区域中，您将看到数字 1，指的是 “行 1”。输入以下 JSON 对象。

```
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani",
  "isbn": "978 158144778"
}
```

- g. 单击**发送**。
- h. 要验证 API 请求是否成功，您现在将看到一个响应，其中包含绿色 **Status** 代码 200 OK。

步骤 5: 使用 **Get /books** API 验证添加的图书。

- a. 返回到第一个 **GET** 选项卡。正如你所看到的，Postman 可以很容易地在不同的 API 调用之间切换。
- b. 单击**发送**。
- c. 要验证 API 请求是否成功，您现在将看到一个响应，其中包含绿色 **Status** 代码 200 OK。
- d. 单击 **正文** 查看响应。请注意，默认值是 **Pretty** 和 **json**。

```
[
  {
    "id": 0,
    "title": "IP Routing Fundamentals",
    "author": "Mark A. Sportack"
  },
  {
    "id": 1,
    "title": "Python for Dummies",
    "author": "Stef Maruch Aahz Maruch"
  },
  {
    "id": 2,
    "title": "Linux for Networkers",
    "author": "Cisco Systems Inc."
  },
  {
    "id": 3,
```

```
"title": "NetAcad: 20 Years Of Online-Learning",
"author": "Cisco Systems Inc."
},
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani"
},
{
  "id": 5,
  "title": "31 Days Before Your CCNA Exam",
  "author": "Allan Johnson"
},
]
```

步骤 6: 将其他参数与 Get /books API 结合使用。

- a. 转到 **School Library API** 网站。向上滚动到 **GET /books** API 并将其展开（如有必要）。

请注意可用的参数：

- **includeISBN**: 在结果中包括 ISBN 编号。Default=false
 - **sortBy**: 使用指定的参数对结果进行排序。Default=id
 - **author**: 只返回给定作者的书籍。
 - **page**: 用于指定页码。
- b. 单击 **Try it out**。如果您看到红色的“**Cancel**”按钮，则无需选择此按钮。
- c. 在参数下：
- 单击“**includeISBN**”并选择“**true**”
 - 单击 **sortBy** 并选择 **author**
- d. 单击**执行 (Execute)**
- e. 在 **响应正文** 中，您将看到现在按作者排序的图书列表，包括 ISBN。

```
[
  {
    "id": 5,
    "title": "31 Days Before Your CCNA Exam",
    "author": "Allan Johnson"
  },
  {
    "id": 2,
    "title": "Linux for Networkers",
    "author": "Cisco Systems Inc.",
    "isbn": "000-0000000123"
  },
  {
    "id": 3,
    "title": "NetAcad: 20 Years Of Online-Learning",
    "author": "Cisco Systems Inc.",

```

```
"isbn": "000-0000001123"
},
{
  "id": 0,
  "title": "IP Routing Fundamentals",
  "author": "Mark A. Sportack",
  "isbn": "978-1578700714"
},
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani",
  "isbn": "978 1587144778"
},
{
  "id": 1,
  "title": "Python for Dummies",
  "author": "Stef Maruch Aahz Maruch",
  "isbn": "978-0471778646"
}
]
```

请注意，**请求 URL** 现在包含参数。你会在 Postman 再次看到这个。

`http://library.demo.local/api/v1/books?includeISBN=true&sortBy=author`

f. 返回 **Postman** 并转到第一个 API 选项卡，GET `http://library.demo.local/api/v1/books`。现在，您将加入 School Library API 网站中的一些参数。

g. 单击 **Params**。您将在 **Query Params** 输入框下看到 **KEY** 和 **VALUE**。输入以下信息：

- 在 “**KEY**” 下，输入 **includeISBN**，并在 “**Value**” 下输入 **true**
请注意，该值的左侧将自动包含一个复选标记，并添加一个新行。
- 在 **KEY** 下，输入 **sortBy**，并在 **Value** 下输入 **author**

请注意，在输入这些查询参数时，它已更新 **GET** 旁边的原始 URL。这与您在 School Library API 网站上看到的相同 API 调用的 **请求 URL** 相同。这是 URL Postman 将在进行 API 调用时使用这些查询参数。

`http://library.demo.local/api/v1/books?includeISBN=true&sortBy=author`

h. 单击**发送**。

注意在 **正文**中，它现在显示相同的书籍列表，按作者排序，包括您在 School Library API 网站中看到的 ISBN。

```
[
{
  "id": 5,
  "title": "31 Days Before Your CCNA Exam",
  "author": "Allan Johnson"
},
{
  "id": 2,
  "title": "Linux for Networkers",
```

```
"author": "Cisco Systems Inc.",
"isbn": "000-0000000123"
},
{
  "id": 3,
  "title": "NetAcad: 20 Years Of Online-Learning",
  "author": "Cisco Systems Inc.",
  "isbn": "000-0000001123"
},
{
  "id": 0,
  "title": "IP Routing Fundamentals",
  "author": "Mark A. Sportack",
  "isbn": "978-1578700714"
},
{
  "id": 4,
  "title": "IPv6 Fundamentals",
  "author": "Rick Graziani",
  "isbn": "978 1587144778"
},
{
  "id": 1,
  "title": "Python for Dummies",
  "author": "Stef Maruch Aahz Maruch",
  "isbn": "978-0471778646"
}
]
```

第 4 部分： 使用 Python 将 100 本书添加到 API 模拟器

您可以使用 OpenAPI 规范试用工具或 Postman 添加任意数量的图书。但是，您必须一次添加一个。更好的解决方案是编写一个程序来添加书籍。在本部分中，您将模拟使用 Python **faker** 库添加 100 本书的过程。

步骤 1： 打开 Visual Studio (VS) 代码并导航到 school-library 目录。

- a. 从 **Menu** 按钮或双击桌面上的图标打开 **VS 代码**。
- b. 单击 **File > Open Folder...**，导航到 **labs/devnet-src/school-library** 文件夹，然后单击 **“OK”**。

步骤 2： 使用 add100RandomBooks.py 程序调查库文件。

- a. 在左侧的 VS 代码 **EXPLORER** 窗格中，单击 **add100RandomBooks.py** 将其打开（如有必要）。
- b. 在顶部，注意将解释器设置为 Python 3 的“shebang”，然后将导入的三个库。

```
#!/usr/bin/env python3

import requests
import json
from faker import Faker
```

- c. 在本课程中，您将使用 Python 请求库。如果您想使用 Python 使用 GET、POST、DELETE 和其他 HTTP 方法发出 API 请求，请求库是必需的。
- d. Faker 是一个为您生成“假”数据的 Python 库。该程序使用 Python **faker** 库生成随机书名、作者和 ISBN。您可以在互联网上搜索更多有关 **faker** 库的信息。但是，请完成以下步骤以查看 **faker** 库的所有 252 种方法。
 - 1) 打开一个终端窗口并启动 Python 3。
 - 2) 从 **faker** 导入 **Faker()** 模块。
 - 3) 将 **Faker()** 模块分配为 **fake** 模块。
 - 4) 要查看所有方法，请输入 **fake**，然后按 Tab 键两次。请注意方法 **fake.name()**，您将在下一步中使用该方法。在下一步中以及本实验的后续步骤中，您还将使用三种突出显示的方法（前面带有 **fake** 的方法）。：**catch_phrase()**、**isbn13()** 和 **name()**。

```
devasc@labvm:~/labs/devnet-src/school-library$ python3
Python 3.8.2 (default, Apr 27 2020, 15:53:34)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from faker import Faker
>>> fake = Faker()
>>> fake.<press Tab key twice - be sure you include the period>
Display all 252 possibilities? (y or n)
fake.add_provider( fake.future_datetime( fake.pyfloat(
fake.address( fake.generator_attrs fake.pyint(
fake.am_pm( fake.get_formatter( fake.pyiterable(
<output omitted>
fake.catch_phrase( fake.ipv4_public( fake.random_element(
fake.chrome( fake.isbn10( fake.random_int(
fake.city( fake.isbn13( fake.random_letter(
<output omitted>
fake.date_time_ad( fake.msisdn( fake.texts(
fake.date_time_between( fake.name( fake.time(
<output omitted>
fake.future_date( fake.pydict( fake.zipcode_plus4(
>>>
```

步骤 3: 练习使用 **faker** 库生成随机数据。

- a. 输入以下内容以生成假名称。每次执行命令时，您的输出将是不同的假名称。

```
>>> print('My name is {}'.format(fake.name()))
My name is Katherine Ross.
>>>
```

- b. 使用上面的步骤 2d 中的三个突出显示的消息，输入打印以下假输出的命令。

```
My name is Gary Castaneda and I wrote "Organic incremental neural-net" (ISBN 978-0-
669-01935-3).
>>>
```

- c. 在程序后面，循环使用了这三种方法，为 School Library 创建条目。输入以下内容以生成 10 个随机名称。在 “...” 之后，您需要再次按回车键。

```
>>> for i in range(10):
...     print(fake.name())
...
Kevin Moyer
Mr. Christopher Green MD
Spencer Jensen
Whitney Guzman
Nicole Scott
Tammy Lewis
Craig Edwards
Michael Diaz
Ryan Mccoy
Terry Rocha
>>>
```

- d. 完成调查 **faker** 库后退出 Python 解释器。

```
>>> quit()
devasc@labvm:~/labs/devnet-src/school-library$
```

步骤 4: 查看函数变量。

程序中的两个函数使用三个变量从 School Library API 服务获取授权令牌。

```
APIHOST = "http://library.demo.local"
LOGIN = "cisco"
PASSWORD = "Cisco123!"
```

步骤 5: 查看 GetAuthToken 函数。

- a. **GetAuthToken** 函数使用三个变量来构建请求。**r** 变量调用 **loginViaBasic** API 的 POST 方法，并在调用成功时存储令牌值。请注意使用 **f** 字符串来构建请求 URL。

```
def getAuthToken():
    authCreds = (LOGIN, PASSWORD)
    r = requests.post(
        f"{APIHOST}/api/v1/loginViaBasic",
        auth = authCreds
    )
```

- b. 如果调用失败（HTTP 状态代码不等于 200），则会引发异常并打印到终端窗口。再次，请注意使用 **f** 字符串来构建异常消息。您可以通过更改步骤 4 中的变量之一来测试异常代码。

```
if r.status_code == 200:
    return r.json()["token"]
else:
    raise Exception(f"Status code {r.status_code} and text {r.text}, while trying to
    Auth.")
```

步骤 6: 查看 “addBook” 功能。

- a. 与 **AddAuthToken** 函数类似，**AddBook** 函数使用步骤 4 中显示的三个变量来构建请求。**r** 变量调用 **books** API 的 POST 方法。数据来自一个名为 **book** 的变量，该变量在程序的最后一部分指定。

```
def addBook(book, apiKey):
    r = requests.post(
        f"{APIHOST}/api/v1/books",
        headers = {
            "Content-type": "application/json",
            "X-API-Key": apiKey
        },
        data = json.dumps(book)
    )
```

- b. 如果调用不成功，则会引发异常并打印到终端窗口。您可以通过在步骤 4 中更改其中一个变量来测试它。

```
if r.status_code == 200:
    print(f"Book {book} added.")
else:
    raise Exception(f"Error code {r.status_code} and text {r.text}, while trying to add
    book {book}.")
```

步骤 7: 查看调用这两个函数的代码。

- a. 添加 **AuthToken** 函数被调用，结果存储在变量 **APIKey** 中。

```
apiKey = getAuthToken()
```

- b. **Faker()** 模块被设置为名为 **fake** 的变量。**for** 循环然后迭代 100 次。**i** 变量稍后在循环中使用，将每本新书的 **id** 键值从 4 到 104 递增，不包括 104。

注意：如果您希望保留之前在本实验中添加的两本书籍，请将范围更改 为 **(6, 106)**。

```
fake = Faker()
for i in range(4, 104):
```

- c. 接下来，三个变量保存从 **Faker()** 模块调用的方法的值：**catch_phrase()**、**name()**和 **isbn13()**。

```
fakeTitle = fake.catch_phrase()
fakeAuthor = fake.name()
fakeISBN = fake.isbn13()
```

- d. 回想一下，**books** API 的有效载荷 参数需要采用以下格式的 JSON：

```
{
    "id": 0,
    "title": "string",
    "author": "string"
}
```

book 变量使用有效载荷参数的三个必需键和三个 **fake** 变量的值构建。

```
book = {"id":i, "title": fakeTitle, "author": fakeAuthor, "isbn": fakeISBN}
```

- e. 最后，**AddBook** 函数被称为传递 **book** 和 **apikey** 变量。由于 **AddBook** 是循环的一部分，因此它将被调用 101 次，书 ID 从 4 递增到 105，不含 105。

步骤 8: 运行并验证 add100RandomBooks.py 程序。

- a. 输入 Python 3 命令以运行 **add100RandomBooks.py** 程序。您应该得到类似于以下内容的输出，尽管您的电子书名和 ISBN 编号将是不同的假值。

```
devasc@labvm:~/labs/devnet-src/school-library$ python3 add100RandomBooks.py
Book {'id': 4, 'title': 'Assimilated client-server frame', 'author': 'Chelsea Mitchell', 'isbn': '978-0-411-83123-3'} added.
Book {'id': 5, 'title': 'Adaptive tangible conglomeration', 'author': 'Edward Ryan', 'isbn': '978-1-64406-014-8'} added.
<output omitted>
Book {'id': 103, 'title': 'Fundamental uniform data-warehouse', 'author': 'Dennis David', 'isbn': '978-1-68465-896-1'} added.
Book {'id': 104, 'title': 'Organic 4thgeneration functionalities', 'author': 'Nicole Gilbert', 'isbn': '978-0-13-176202-2'} added.
devasc@labvm:~/labs/devnet-src/school-library$
```

- b. 返回 Chromium 浏览器并刷新 **http://library.demo.local/** 网页。现在，您应该看到添加了 100 本新书。

注意：如果您进入 API 文档页面而不是主页面（**http://library.demo.local/api/v1/docs**）并使用“Try it out”，则只会获得前 10 本书的列表。您可以输入一个值从 2 到 10 的 **页面** 参数，以查看其他书籍。

您将如何添加另外 100 本书？